FATECalçados — Guia do Projeto

Pastas, funções e endpoints · Desafio de Inovação Data System 2026

Programa de fidelidade da FATECalçados: um **backend** Django + PostgreSQL que guarda as regras e os dados, e um **frontend** React + Vite com o caixa (PDV), o painel da gerência (Dashboard, Clientes, Vendas, Ações) e a área do cliente (loja online e Minha conta). Tudo sobe com `docker compose up -d --build`; a interface fica em http://localhost:5173 e o admin em http://localhost:8000/admin.

1. Estrutura de pastas

Pasta / arquivo	Para que serve
`docker-compose.yml`	Sobe os três serviços: db (Postgres 16), backend (Django, porta 8000) e frontend (Vite, porta 5173).
`docs/`	Documentos do projeto em PDF (regras de pontos, onde usamos IA e este guia).
`backend/config/`	Projeto Django: settings.py (banco, CSRF, apps), urls.py raiz (admin + rotas da app), asgi/wsgi.
`backend/src/`	A única app Django: modelos, regras, serviços, consultas e views da API.
`backend/src/migrations/`	Migrações do banco (0001 inicial, 0002 PDV, 0003 acesso do cliente).
`backend/src/management/commands/`	Comandos manage.py: popular_dados (seed) e expirar_pontos.
`backend/src/tests/`	Testes Django (regras, PDV, consultas, ações, conta, telas).
`frontend/src/routes/`	Rotas do React Router (createBrowserRouter).
`frontend/src/layouts/`	MainLayout (telas da loja, com sidebar e tour) e ClienteLayout (área do cliente, exige sessão).
`frontend/src/pages/`	Uma subpasta por área: inicio, pdv, dashboard, clientes, vendas, acoes, conta.
`frontend/src/components/ui/`	Peças visuais reaproveitadas: Button, Card, Modal, Pill, Led, LevelBadge, Pagination, Toast…
`frontend/src/components/layout/`	Sidebar, navegação (navegacao.ts), busca Ctrl/Cmd K, tour guiado e painel de acessibilidade.
`frontend/src/components/charts/`	Gráfico de pontos do dashboard (PointsChart).
`frontend/src/contexts/`	Estado que sobrevive à troca de página: Caixa, Ações, Conta e Toast (contexto + Provider).
`frontend/src/hooks/`	Lógica das telas (carrinho, fila offline, atalhos, tema, tour, carregamento da API…).
`frontend/src/services/api/`	Única camada que chama a API; um módulo por área, traduzindo campos PT -> EN.
`frontend/src/services/filaVendas.ts`	Fila de vendas offline guardada no localStorage (fatec:vendas).
`frontend/src/utils/`	Funções puras usadas em 2+ lugares: format.ts (dinheiro, datas) e cpf.ts.
`frontend/src/regras.ts`	Espelho das regras de pontos para as prévias do caixa (o backend manda).
`frontend/src/config.ts`	Qual caixa este PDV é (REGISTER_CODE).

2. Backend (backend/src/)

As camadas são rígidas e cada uma só chama a de baixo: **views** -> **services / consultas / indicadores / acoes** -> **regras** (funções puras). Os pontos são um livro-razão: o saldo é sempre a soma dos `MovimentoPontos`, sem campo de saldo guardado.

models.py — tabelas

Modelo	Campos principais
`Loja`	nome, cidade
`Caixa`	codigo (ex.: araras-01), nome, loja
`Produto`	sku, nome, preco, ativo
`Cliente`	cpf (só números), nome, cadastrado_em, loja_cadastro, usuario (login da área do cliente)
`Compra`	id (UUID do caixa), cliente, loja, caixa, forma_pagamento, data, valor_bruto, pontos_resgatados, desconto, valor_pago, nivel_na_compra, pontos_ganhos
`ItemCompra`	compra, produto, quantidade, preco_unitario
`MovimentoPontos`	cliente, compra, tipo (acúmulo/resgate/expiração), pontos, data, expira_em, restante (do lote)

regras.py — regras do programa (sem banco)

Níveis por pontos acumulados em 12 meses: Bronze (0, +0%), Prata (1.000, +10%), Ouro (5.000, +20%). Resgate em blocos de 100 pontos = R$ 5,00. Pontos valem 12 meses.

Função	O que faz
`nivel_por_pontos(pontos_12m)`	Nível correspondente aos pontos acumulados nos últimos 12 meses.
`nome_do_nivel(codigo)`	Nome de exibição do nível (BRONZE -> Bronze).
`bonus_do_nivel(codigo)`	Percentual de bônus do nível (0, 10 ou 20).
`calcular_pontos(valor_pago, nivel)`	Pontos da compra: reais pagos sem centavos x (100 + bônus) / 100, arredondando para baixo.
`desconto_do_resgate(pontos)`	Valor em R$ que os pontos resgatados descontam.
`proximo_nivel(pontos_12m)`	Nome do próximo nível e quantos pontos faltam (None no Ouro).
`faixa_do_nivel(codigo)`	Faixa mínimo/limite de pontos do nível, usada para filtrar clientes no banco.

services.py — única camada que escreve regras no banco

Função	O que faz
`RegraViolada`	Exceção de regra de negócio; as views devolvem 400 com {"erro": ...}.
`saldo(cliente, ate)`	Soma dos movimentos de pontos até a data.
`pontos_12m(cliente, em)`	Pontos acumulados na janela de 12 meses.
`nivel_atual(cliente, em)`	Nível atual a partir de pontos_12m.
`cadastrar_cliente(cpf, nome, loja, data)`	Valida o CPF e cria o cliente.
`expirar_pontos(cliente, ate)`	Lança a expiração dos lotes vencidos e devolve os pontos expirados.
`_consumir_lotes(cliente, pontos, em)`	Baixa pontos dos lotes mais antigos primeiro (FIFO).
`registrar_compra(...)`	Aplica resgate, desconto, nível de antes da compra e cria o lote de pontos.
`buscar_produtos(texto)`	Produtos ativos filtrados por nome ou SKU.
`registrar_venda(...)`	Venda do caixa: preços do banco, itens, idempotência pelo UUID; chama registrar_compra.
`caixa_online()`	Cria sob demanda o caixa online-01 da "Loja Online".
`criar_acesso(cpf, nome, senha)`	Cria o login do cliente (novo CPF ou existente com o mesmo primeiro nome).
`autenticar_cliente(request, cpf, senha)`	Faz login por CPF + senha e abre a sessão.
`comprar_online(cliente, itens, ...)`	Compra da loja online pelo mesmo registrar_venda, no caixa online-01.

Função	O que faz
`alterar_senha(cliente, atual, nova)`	Troca a senha conferindo a atual.

consultas.py — listas e perfis (somente leitura)

Função	O que faz
`listar_clientes(busca, nivel, pagina)`	Lista paginada de clientes com saldo, nível e CPF mascarado.
`perfil_do_cliente(cliente)`	Saldo, nível, progresso, compras e extrato de pontos do cliente.
`listar_vendas(inicio, fim, caixa, pagamento, pagina)`	Lista paginada de vendas com filtros.
`detalhe_da_venda(compra)`	Itens, valores e pontos de uma venda.
`painel_do_cliente(cliente)`	Dados da área "Minha conta".
`_lotes_validos / _paginar`	Auxiliares: saldo pelos lotes ainda válidos (sem escrever) e paginação.

indicadores.py — números do Dashboard (somente leitura)

Função	O que faz
`periodo_padrao / periodo_de_datas`	Período padrão e conversão de datas da URL.
`base_do_programa(ref)`	Clientes por nível e base ativa.
`pontos_e_custo(inicio, fim)`	Pontos ganhos, resgatados, expirados e o custo em R$.
`ranking(inicio, fim)`	Top clientes do período.
`evolucao_mensal(inicio, fim)`	Série mensal de pontos para o gráfico.
`resumo_de_vendas / vendas_por_caixa / vendas_por_pagamento`	Faturamento, ticket médio e distribuição das vendas.
`produtos_mais_vendidos(inicio, fim)`	Top produtos por quantidade.
`resgate_por_nivel(inicio, fim)`	Quanto cada nível resgata.
`pontos_a_vencer(ref, dias)`	Pontos que vencem nos próximos dias (passivo).

acoes.py — listas da aba Ações

Função	O que faz
`clientes_sumindo(ref)`	Clientes sem comprar há 60+ dias.
`pontos_vencendo(ref)`	Clientes com pontos vencendo em até 30 dias.
`perto_de_subir(ref)`	Clientes a até 300 pontos do próximo nível.
`novos_sem_segunda_compra(ref)`	Clientes novos sem segunda compra após 30 dias.
`acoes_do_dia(ref)`	Junta as quatro listas na resposta de /api/acoes/.

Outros arquivos

Arquivo	O que faz
`cpf.py`	limpar, validar (dígitos verificadores), formatar, mascarar (***.456.789-**) e gerar CPFs.
`views.py`	Funções que leem a requisição, chamam os serviços e devolvem JSON.

Arquivo	O que faz
admin.py	Admin de Loja, Caixa, Produto e Cliente; Compra e MovimentoPontos são somente leitura.
popular_dados	Apaga e recria dados fictícios (--clientes, --compras, --sem-cpf, --meses, --semente), sempre por registrar_venda.
expirar_pontos	Expira os lotes vencidos de todos os clientes.

3. Endpoints da API

Todas as respostas são JSON. Erros de regra voltam como 400 com `{"erro": "..."}`. As rotas da loja não usam sessão (csrf_exempt); as de `/api/conta/` usam o cookie de sessão e exigem o cabeçalho X-CSRFToken.

Método	Rota	Descrição
GET	/api/dashboard/	Indicadores do período (?inicio=&fim=).
GET	/api/acoes/	Listas de ações do dia para a gerência.
GET	/api/produtos/	Produtos ativos (?busca=).
GET	/api/clientes/	Lista paginada (?busca=&nivel=&pagina=).
POST	/api/clientes/	Cadastra cliente {cpf, nome, caixa}.
GET	/api/clientes/<cpf>/	Busca o cliente pelo CPF no caixa (saldo e nível).
GET	/api/clientes/<id>/perfil/	Perfil completo do cliente.
GET	/api/vendas/	Lista paginada (?inicio=&fim=&caixa=&pagamento=&pagina=).
POST	/api/vendas/	Registra venda {id, caixa, itens[sku, quantidade], forma_pagamento, cpf, pontos_resgate, data}. Reenvio do mesmo id devolve 200.
GET	/api/vendas/<uuid>/	Detalhe de uma venda.
GET	/api/conta/	Painel do cliente logado (401 sem sessão).
POST	/api/conta/cadastro/	Cria acesso {cpf, nome, senha} e já entra.
POST	/api/conta/entrar/	Login {cpf, senha}.
POST	/api/conta/sair/	Encerra a sessão.
POST	/api/conta/senha/	Troca a senha {senha_atual, nova_senha}.
POST	/api/conta/compras/	Compra online {id, itens, forma_pagamento, pontos_resgate}; CPF vem da sessão.
GET	/api/conta/compras/<uuid>/	Detalhe de uma compra do próprio cliente.
—	/admin/	Admin do Django (precisa de createsuperuser).

4. Frontend (frontend/src/)

Telas (pages/)

Pasta	Rota	Conteúdo
inicio/	/, /login, /cadastro	Landing com vitrine de tênis, telas de acesso do cliente e botão de tema.
pdv/	/pdv	Caixa: busca e grade de produtos, carrinho, cliente, pontos, pagamento, barra de conexão.
dashboard/	/dashboard	Indicadores em seções (Pontos, Vendas, Clientes), filtro de período, exportar CSV/PDF.

Pasta	Rota	Conteúdo
clientes/	/clientes, /clientes/:id	Lista com filtros na URL e perfil do cliente.
vendas/	/vendas	Lista de vendas com filtros e modal de detalhe.
acoes/	/acoes	Cartões de ações da gerência, marcáveis como feitas.
conta/	/loja, /loja/carrinho, /minha-conta…	Loja online, checkout, compras, pontos e dados do cliente.

services/api/ — chamadas à API

Função	Endpoint
request, errorMessage, queryString (http.ts)	Base: CSRF, erros como ApiError e query string.
fetchProducts	GET /api/produtos/
fetchClient, registerClient	GET /api/clientes/<cpf>/ · POST /api/clientes/
fetchClients, fetchClientProfile	GET /api/clientes/ · GET /api/clientes/<id>/perfil/
sendSale, fetchSales, fetchSaleDetail	POST e GET /api/vendas/ · GET /api/vendas/<uuid>/
fetchActions	GET /api/acoes/
fetchDashboard	GET /api/dashboard/
fetchAccount, signIn, createAccess, signOut	/api/conta/, entrar/, cadastro/, sair/
buyOnline, fetchAccountPurchase, changePassword	/api/conta/compras/, compras/<uuid>/, senha/

Hooks (hooks/)

Hook	O que faz
useCarrinho / useCarrinhoOnline	Itens e totais do carrinho do caixa e da loja online.
useFilaVendas	Fila offline de vendas e reenvio ao reconectar.
useCarregar / useDashboard	Carrega da API com estados de carregando, erro e tentar de novo.
useAtalhosPdv / useAtalhoBusca	Atalhos F2/F10 no caixa e Ctrl/Cmd K para a busca.
usePeriodoNaUrl	Período do dashboard guardado na URL.
useSidebar / useTema / useAcessibilidade	Sidebar aberta, tema claro/escuro e opções de acessibilidade.
useTour	Tour guiado da primeira visita.
useTemaClaroAoImprimir	Força o tema claro ao exportar PDF.
useVitrine / useProgressoScroll / useDebounce / useTituloDaRota	Vitrine da landing, progresso de rolagem, debounce e título da página.

Contextos, regras e utilitários

Item	O que faz
CaixaProvider / useCaixa	Online/offline, fila de vendas e venda em andamento.
AcoesProvider / useAcoes	Listas de ações e quais foram feitas (badge da sidebar).
ContaProvider / useConta	Sessão do cliente e carrinho online.
ToastProvider / useToast	Avisos rápidos com notify(mensagem).
regras.ts	pointsToMoney, maxRedeemPoints e estimatePoints para as prévias do caixa.

Item	O que faz
`utils/format.ts`	formatMoney, formatNumber, formatPercent, formatDate, plural, normalizeText…
`utils/cpf.ts`	onlyDigits, formatCpf e isValidCpf (espelho do cpf.py).
`services/filaVendas.ts`	loadSales, saveSales, sendAndMark e sendPendingSales.

5. Comandos úteis

Comando	Para que serve
`docker compose up -d --build`	Sobe tudo.
`docker compose exec backend python manage.py migrate`	Cria/atualiza as tabelas.
`docker compose exec backend python manage.py popular_dados`	Gera dados de exemplo.
`docker compose exec backend python manage.py expirar_pontos`	Expira pontos vencidos.
`docker compose exec backend python manage.py test`	Testes do backend.
`npm run lint · npm run build · npm test (em frontend/)`	Lint, build e testes do frontend.

Atenção: nunca rode `docker compose down -v`: apaga o volume do banco.